Spring Data Elasticsearch

If you are working on a Spring framework-based Java application, there's another route to integrate it with Elasticsearch: Spring Data Elasticsearch. This project helps us connect and query Elasticsearch using familiar and well-proven Spring patterns such as templates and repositories. Just as we'd create a repository to represent a database layer, Spring Data Elasticsearch lets us use the repository layer to query Elasticsearch. Refer to the documentation for more on the Spring Data Elasticsearch project: http://mng.bz/vnyr.

C.2 Background

Elasticsearch released a new forward-compatible Java client, Java API Client, from version 7.17. The Java API client is a modern client that follows a decent pattern of feature clients with strongly typed requests and responses. The earlier incarnation of the client was Java High-Level REST Client (see notes below)—which attracted a fair amount of criticism as well as became a headache to maintain and manage for elastic folks due to its inherent nature of how it was designed by having a dependency on the shared common code with Elasticsearch server.

Elastic folks recognized the need for a modern client—independent of the server's code base, with its client API code generated based on the server's API schema and providing a "feature client" pattern (we see this in action shortly). This led to the creation of a Java API client. It is the next-generation lightweight client whose code is pretty much (99%!) generated, embracing fluent API builder patterns and auto marshaling and unmarshalling Java Objects to JSON and vice versa.

Java High-Level REST Client

Elasticsearch re-engineered to get away with Java High-Level REST client (let me put an acronym on this for easier reference later on JHLRC) and introduced an API-based client called Java API client. There are a few problems with the JHLRC. In particular, it is coupled with specific versions of the Elasticsearch server because it shares common code with the Elasticsearch server. APIs are all manually handwritten, which means a lot of maintenance over time, leading to it becoming error-prone. The dependency on the server codebase means backward/forward compatibility is compromised.

C.3 Maven/Gradle project setup

The `ElasticsearchClient`-related classes are provided in a jar artifact that can be downloaded as part of the dependencies in our project. For convenience, I've created a Maven-based project that is available on GitHub (http://mng.bz/4Dmv) and the book's website.

Usually, two dependencies required to bring in related classes. They are declared in the pom file.